

Improve default container formatting

Document #: P2585R0
Date: 2022-05-14
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[2 Design](#)

[3 Wording](#)

[3.1 Trait and Regular Sequences](#)

[3.2 Maps and sets](#)

[3.3 Strings](#)

[4 Acknowledgements](#)

[5 References](#)

§ 1 Introduction

[P2286R8] adds support for formatting any range whose underlying type is formattable. Additionally, it adds support for different kinds of formatting that users can opt into, while also providing a default choice for associating containers that is more suited to what those containers represent.

For example, simply using "{}" as the specifier, we get the following outputs:

Expression	Output
<code>std::vector<std::pair<int, int>>{{1, 2}, {3, 4}}</code>	<code>[(1, 2), (3, 4)]</code>
<code>std::set<std::pair<int, int>>{{1, 2}, {3, 4}}</code>	<code>{{(1, 2), (3, 4)}}</code>
<code>std::map<int, int>{{1, 2}, {3, 4}}</code>	<code>{1: 2, 3: 4}</code>

In each case, we have a range over a pair of ints, but we have three different outputs - as appropriate for the different kinds of containers.

However, this distinction is a result of [P2286R8] explicitly providing formatters for all the standard library map and set containers, and applying those changes to them. This is something that users *can* do for their own containers as well, but which also means that it is something users *have* to do - if this is the behavior they want. For instance, the containers in Boost or Abseil just format like normal ranges:

Expression	Output
<code>boost::container::flat_set<int>{1, 2, 3}</code>	<code>[1, 2, 3]</code>
<code>absl::flat_hash_map<int, int>{{1, 2}, {3, 4}}</code>	<code>[(1, 2), (3, 4)]</code>

This output isn't *wrong*, per se. It's just that it's not ideal. And, sure, Abseil could certainly add the code necessary to make this happen. Which, at a bare minimum, would be:

```
template <class K, class V, class... Rest>
    requires std::formattable<const K, char>
        && std::formattable<const V, char>
struct std::formatter<absl::flat_hash_map<K, V, Rest...>, char>
    : std::range_formatter<std::pair<const K, V>, char>
{
    constexpr formatter() {
        this->set_brackets("{", "}");
        this->underlying().set_brackets({}, {});
        this->underlying().set_separator(": ");
    }
};
```

Now, this isn't a lot of code, nor is it especially complex. But it is writing code to do what seems like it should be the default behavior.

And, indeed, this is what `{fmt}` does. By default, formatting a map (whether it's a `std::map` or an `absl::flat_hash_map`) does give you a string like `{1: 2, 3: 4}` rather than `[(1, 2), (3, 4)]`. `{fmt}` determines whether a type is map-like simply by checking if it has a member type named `mapped_type` ([here](#)). Similarly, `{fmt}` determines whether a type is set-like if it has a member type named `key_type` (and it is not map-like) ([here](#)).

We can do something similar in the standard library.

§ 2 Design

There are several kinds of range formatting styles:

- map-like, as `{k1: v1, k2: v2}`
- set-like, as `{v1, v2}`
- regular sequence, as `[v1, v2]`
- string, as `abc`
- debug string, `"\naan"`
- disabled

We can introduce an enum class for all of these kinds:

```
enum class range_format_kind {
    disabled,
    map,
```

```

    set,
    sequence,
    string,
    debug_string,
};

```

Note that `range_format_kind::disabled` is `0` so that converting to `bool` checks if it's enabled.

And a variable template (allowed to be specialized) to determine the right kind:

```

template <class R>
inline constexpr auto format_kind = unspecified;

template <input_range R>
inline constexpr range_format_kind format_kind<R> = []{
    if constexpr (requires { typename R::key_type; typename R::mapped_type; }
        and is-2-tuple<range_reference_t<R>>()) {
        return range_format_kind::map;
    } else if constexpr (requires { typename R::key_type; }) {
        return range_format_kind::set;
    } else {
        return range_format_kind::sequence;
    }
}();

```

`format_kind<U>` for non-range types `U` is *unspecified* to allow for future evolution in case we want to do this for non-range types. For now, it doesn't cost anything.

As far as heuristics go, this is a pretty safe bet. While we've previously tried to do a heuristic for `view` and then had to walk it back a lot, here the stakes are much lower. Worst-case, we're just talking about getting the default formatting wrong - for some definition of wrong - though we'd still format all the elements. And there probably aren't too many range types floating around that define `key_type` and `mapped_type` but aren't maps? Note that we're being especially safe here by checking not just `R::key_type` and `R::mapped_type` but also that the reference type is a two-tuple (by which I mean that it's either a specialization of `std::pair` or a specialization of `std::tuple` whose size is `2`). This added check ensures that we don't accidentally end up with an ill-formed default whereas the previous default could have worked fine (on the off chance that we have some type with a `key_type` and a `mapped_type` yet is somehow a range of `int`).

Note that we're deliberately not making any sort of guess about `string`. Not looking at convertibility to `string` or `string_view` or `char const*` or anything of the sort. If users want some container to be formatted as a string, they can do so by explicitly specializing `format_kind` - which isn't that much effort at all. The problem is that coming up with a heuristic would complicate `format_kind`, since we can't just check convertibility to `string_view`, we have to specifically check convertibility to `basic_string_view<charT>`, and now suddenly we have this extra `charT` parameter. Is it worth it? I'm not sure it is.

If we then change the way the standard library does formatting based on `format_kind<R>`, then with no additional code changes necessary, all the Boost and Abseil containers would start being formatted in the

same kind of way that the standard library ones are today. That and many other user-defined associative containers.

Moreover, for user-defined containers, specializing a variable template is quite a bit less work than almost anything else you can do. So even for containers where the heuristic is wrong, this design offers an easier way forward to get the desired behavior.

While this design as-is has not been implemented, `{fmt}`'s approach is pretty comparable.

§ 3 Wording

This is in terms of [P2286R8]. The wording here chooses to define `formatter<R, charT>` as inheriting from an exposition-only `default-range-formatter<format_kind<R>, R, charT>` rather than a bunch of specializations for each `format_kind<R>` case because this allows future extensions where uses can create more constrained specializations of `formatter` for enabled ranges much easier.

§ 3.1 Trait and Regular Sequences

Change 22.14.1 [format.syn]:

```
namespace std {
    // ...

    // [format.formatter], formatter
    template<class T, class charT = char> struct formatter;

    // [format.range.formatter], class template range_formatter
+ enum class range_format_kind {
+     disabled,
+     map,
+     set,
+     sequence,
+     string,
+     debug_string
+ };
+
+ template<class R>
+     inline constexpr unspecified format_kind = unspecified;
+
+ template<ranges::input_range R>
+     inline constexpr range_format_kind format_kind<R> = see below;

    template<class T, class charT = char>
        requires same_as<remove_cvref_t<T>, T> && formattable<T, charT>
        class range_formatter;

+ template<range_format_kind K, ranges::input_range R, class charT>
+     struct default_range_formatter; // exposition only

    template<ranges::input_range R, class charT>
```

```

-     requires (!same_as<remove_cvref_t<ranges::range_reference_t<R>>,
R>)
+     requires (format_kind<R> != range_format_kind::disabled)
           && formattable<ranges::range_reference_t<R>, charT>
-     struct formatter<R, charT>;
+     struct formatter<R, charT> : default-range-formatter<format_kind<R>, R,
charT> { };

// ...
}

```

Add to [format.range]:

```

template<input_range R>
    inline constexpr range_format_kind format_kind<R> = see below;

```

a For a type *R*, `format_kind<R>` is defined as follows:

- (a.1) — If `same_as<remove_cvref_t<ranges::range_reference_t<R>>, R>` is `true`, `format_kind<R>` is `range_format_kind::disabled`. [Note: This prevents constraint recursion for ranges whose reference type is the same range type. For example, `std::filesystem::path` is a range of `std::filesystem::path`. -end note]
- (a.2) — Otherwise, if the *qualified-id* `R::key_type` is valid and denotes a type:
 - (a.2.1) — If the *qualified-id* `R::mapped_type` is valid and denotes a type, let *U* be `remove_cvref_t<ranges::range_reference_t<R>>`. If either *U* is a specialization of `pair` or *U* is a specialization of `tuple` and `tuple_size_v<U> == 2`, `format_kind<R>` is `range_format_kind::map`.
 - (a.2.2) — Otherwise, `format_kind<R>` is `range_format_kind::set`.
- (a.3) — Otherwise, `format_kind<R>` is `range_format_kind::sequence`.

b *Remarks:* Pursuant to [namespace.std], users may specialize `format_kind` for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions ([`expr.const`]) and have type `const range_format_kind`.

And later:

```

namespace std {
- template<ranges::input_range R, class charT>
-     requires (!same_as<remove_cvref_t<ranges::range_reference_t<R>>,
R>)
-     && formattable<ranges::range_reference_t<R>, charT>
-     struct formatter<R, charT> {
+ template <ranges::input_range R, class charT>
+ struct default-range-formatter<range_format_kind::sequence, R, charT> {
    private:
        using maybe-const-r = fmt-maybe-const<R, charT>;
        range_formatter<remove_cvref_t<ranges::range_reference_t<maybe-const-
r>>, charT> underlying_; // exposition only

    public:
        constexpr void set_separator(basic_string_view<charT> sep);

```

```

constexpr void set_brackets(basic_string_view<charT> opening,
    basic_string_view<charT> closing);

template <class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);

template <class FormatContext>
typename FormatContext::iterator
    format(maybe-const-r& elems, FormatContext& ctx) const;
};
}

```

- 13 ~~[Note: The `(!same_as<remove_cvref_t<ranges::range_reference_t<R>>, R>)` constraint prevents constraint recursion for ranges whose reference type is the same range type. For example, `std::filesystem::path` is a range of `std::filesystem::path`. —end note]~~

```
constexpr void set_separator(basic_string_view<charT> sep);
```

- 14 Effects: Equivalent to `underlying_.set_separator(sep);`

§ 3.2 Maps and sets

Change the the wording for associative containers as follows:

- 1 ~~For each of `map`, `multimap`, `unordered_map`, and `unordered_multimap`, the library provides the following formatter specialization where `map-type` is the name of the template:~~

```

namespace std {
- template <class charT, class Key, formattable<charT> T, class... U>
-     requires formattable<const Key, charT>
-     struct formatter<map-type<Key, T, U...>, charT>
+ template <ranges::input_range R, class charT>
+     struct default_range_formatter<range_format_kind::map, R, charT> {
    {
    private:
-     using maybe-const-map = fmt-maybe-const<map-type<Key, T, U...>, charT>;
-         // exposition only
-     range_formatter<remove_cvref_t<ranges::range_reference_t<maybe-const-
-         map>>, charT> underlying_; // exposition only
+     using maybe-const-map = fmt-maybe-const<R, charT>;
+         // exposition only
+     using element-type = remove_cvref_t<ranges::range_reference_t<maybe-
+         const-map>>; // exposition only
+     range_formatter<element-type, charT> underlying_;
+         // exposition only
    public:
        constexpr formatter();

        template <class ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);
    };
}

```

```

    template <class FormatContext>
        typename FormatContext::iterator
        format(maybe-const-map& r, FormatContext& ctx) const;
};
}

```

```
constexpr formatter();
```

2 *Mandates:* Either:

(2.1) — *element-type* is a specialization of `pair`, or

(2.2) — *element-type* is a specialization of `tuple` and `tuple_size_v<T> == 2`.

3 *Effects:* Equivalent to:

```

underlying_.set_brackets(STATICALLY-WIDEN<charT>("{"), STATICALLY-
    WIDEN<charT>("}"));
underlying_.underlying().set_brackets({}, {});
underlying_.underlying().set_separator(STATICALLY-WIDEN<charT>(": "));

```

```

template <class ParseContext>
    constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);

```

4 *Effects:* Equivalent to `return underlying_.parse(ctx);`

```

template <class FormatContext>
    typename FormatContext::iterator
    format(maybe-const-map& r, FormatContext& ctx) const;

```

5 *Effects:* Equivalent to `return underlying_.format(r, ctx);`

6 ~~For each of `set`, `multiset`, `unordered_set`, and `unordered_multiset`, the library provides the following `formatter` specialization where `set-type` is the name of the template:~~

```

namespace std {
- template <class charT, class Key, class... U>
-     requires formattable<const Key, charT>
- struct formatter<set-type<Key, U...>, charT>
+ template <ranges::input_range R, class charT>
+ struct default-range-formatter<range_format_kind::set, R, charT> {

    {
    private:
-     range_formatter<Key, charT> underlying_; // exposition only
+     using maybe-const-set = fmt-maybe-const<R, charT>;
        // exposition only
+     range_formatter<remove_cvref_t<ranges::range_reference_t<maybe-const-
        set>>, charT> underlying_; // exposition only

    public:
        constexpr formatter();

        template <class ParseContext>
            constexpr typename ParseContext::iterator

```

```

        parse(ParseContext& ctx);

    template <class FormatContext>
        typename FormatContext::iterator
-       format(const set-type<Key, U...>& r, FormatContext& ctx) const;
+       format(maybe-const-set& r, FormatContext& ctx) const;
    };
}

```

```
constexpr formatter();
```

- 7 *Effects:* Equivalent to:

```
underlying_.set_brackets(STATICALLY-WIDEN<charT>("{"), STATICALLY-
                        WIDEN<charT>("}"));
```

```

template <class ParseContext>
    constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);

```

- 8 *Effects:* Equivalent to `return underlying_.parse(ctx);`

```

template <class FormatContext>
    typename FormatContext::iterator
-   format(const set-type<Key, U...>& r, FormatContext& ctx) const;
+   format(maybe-const-set& r, FormatContext& ctx) const;

```

- 9 *Effects:* Equivalent to `return underlying_.format(r, ctx);`

§ 3.3 Strings

Also add this partial specialization to handle string types:

```

template <range_format_kind K, ranges::input_range R, class charT>
    requires (K == range_format_kind::string ||
              K == range_format_kind::debug_string)
struct default-range-formatter<K, R, charT> {
private:
    formatter<basic_string<charT>, charT> underlying_; // exposition only

public:
    template <class ParseContext>
        constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

    template <class FormatContext>
        typename FormatContext::iterator
        format(see below& str, FormatContext& ctx) const;
};

```

- 1 *Mandates:* `same_as<remove_cvref_t<range_reference_t<R>>, charT>` is true.

```

template <class ParseContext>
    constexpr typename ParseContext::iterator

```



```
    parse(ParseContext& ctx);
```

2 *Effects*: Equivalent to:

```
    auto i = underlying_.parse(ctx);  
    if constexpr (K == range_format_kind::debug_string) {  
        underlying_.set_debug_format();  
    }  
    return i;
```

```
template <class FormatContext>  
    typename FormatContext::iterator  
        format(see below& r, FormatContext& ctx) const;
```

3 The type of *r* is `const R&` if `input_range<const R> is true` and `R&` otherwise.

4 *Effects*: Equivalent to `return underlying_.format(basic_string<charT>(from_range, r), ctx);`

§ 4 Acknowledgements

Thanks to Jeff Garland for initially pointing out this distinction, Tomasz Kamiński for suggesting this approach, Victor Zverovich for having more or less already implemented it, and Tim Song for everything else.

§ 5 References

[P2286R8] Barry Revzin. 2022-05-13. Formatting Ranges.
<https://wg21.link/p2286r8>